Musterlösung zu Übung 2

CSS Grundlagen

Einführung in Webengineering · WS 2026/27

Diese Musterlösung erläutert die zehn Aufgaben und zeigt mögliche Umsetzungen. Vergleichen Sie Ergebnisse und Lösungswege. Andere fachlich richtige Gestaltungen sind ebenfalls möglich.

01 Inhalt und Gestaltung trennen

a) Aufgaben der drei Sprachen

HTML beschreibt Inhalt, Struktur und Bedeutung, etwa Überschriften und Links. **CSS** legt die Darstellung fest, etwa Farben, Abstände und Anordnung. **JavaScript** ermöglicht programmiertes Verhalten, etwa Reaktionen auf Eingaben oder das Nachladen von Inhalten.

Eine gemeinsame externe CSS-Datei bietet zwei Vorteile: Sie gestaltet mehrere Seiten einheitlich und ermöglicht zentrale Änderungen. Eine angepasste Regel wirkt auf alle Seiten, die die Datei einbinden.

b) Aufbau der CSS-Regel

```
h1, h2 {
  color: #154a6c;
  border-bottom: 2px solid currentColor;
}
```

Die Selektoren `h1` und `h2` bestimmen die betroffenen Elemente. Das Komma trennt die Selektoren der Liste: Die Regel gilt für beide Überschriftentypen.

`color: #154a6c` und `border-bottom: 2px solid currentColor` sind Deklarationen. `color` und `border-bottom` sind Eigenschaften; `#154a6c` und `2px solid currentColor` sind ihre Werte. Der Doppelpunkt trennt Eigenschaft und Wert, das Semikolon trennt Deklarationen. Das letzte Semikolon vor `}` darf entfallen, ist aber sinnvoll. Die geschweiften Klammern begrenzen den Deklarationsblock.

`currentColor` verwendet den berechneten Wert von `color`. Der untere Rahmen ist daher hier ebenfalls dunkelblau und zwei CSS-Pixel breit.

c) Externe Datei und die beiden Alternativen

Die Dateien `index.html` und `material.html` liegen in `kapitel02`. Die Regel aus b) steht in `kapitel02/css/styles.css`. Beide HTML-Dateien binden sie im `head` ein:

```
<link rel="stylesheet" href="css/styles.css">
```

Beide Seiten zeigen nun farbige `h1`- und `h2`-Überschriften mit unterem Rahmen. Der relative Pfad wird jeweils von der HTML-Datei aus aufgelöst.

Alternativ lässt sich die Textfarbe unmittelbar am Element setzen:

```
<h2 style="color: #154a6c">Unsere Themen</h2>
```

Oder über ein `style`-Element im `head` des Dokuments:

```
<style>
  h2 { color: #154a6c; }
</style>
```

02 Selektoren und Kombinatoren anwenden

a) Ausgewählte Elemente

Ausgewählt werden die Elemente mit den folgenden IDs:

Selektor	IDs	Begründung
`.info`	`p1, p3`	Beide Absätze besitzen die Klasse `info`.
`section p`	`p1, p2, p3`	Alle drei Absätze sind Nachfahren des Abschnitts.
`section > p`	`p1, p3`	Nur diese Absätze sind direkte Kinder des Abschnitts.
`h2 + p`	`p1`	Unmittelbar folgendes Geschwisterelement der Überschrift.
`h2 ~ p`	`p1, p3`	Nachfolgende Absätze mit demselben Elternelement wie die Überschrift.
`[href$=".pdf"]`	`a1`	Der Wert des Attributs `href` endet mit `.pdf`.

Das **Leerzeichen** erlaubt beliebig viele Zwischenebenen; `>` verlangt eine direkte Eltern-Kind-Beziehung. `+` verlangt das nächste Geschwisterelement; `~` erlaubt weitere dazwischenliegende Geschwisterelemente. Beide Geschwisterkombinatoren wählen nur nachfolgende Elemente aus. Leerraum und Kommentare zwischen den Elementen ändern diese Beziehungen nicht.

b) Eigene Selektoren

```
#p2 { border: 2px solid red; }
.info.extra { border: 2px solid red; }
a[href^="https://"] { border: 2px solid red; }
```

`.info.extra` verlangt beide Klassen am selben Element. `.info .extra` würde dagegen nach einem Nachfahren mit der Klasse `extra` suchen. Der HTTPS-Selektor hat im vorgegebenen Ausschnitt keinen Treffer; ein passender Link kann zum Test ergänzt werden.

c) Vollständiges Testdokument

```
<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <title>Selektoren testen</title>
  <style>section > p { border: 2px solid red; }</style>
</head>
<body>
  <section id="kurs">
    <h2>Themen</h2>
    <p id="p1" class="info">HTML</p>
    <div><p id="p2">CSS</p></div>
    <p id="p3" class="info extra">Übung</p>
    <a id="a1" href="material.pdf">Materialien</a>
  </section>
</body>
</html>
```

Wird nur `section > p` durch `section p` ersetzt, erhält zusätzlich `p2` einen Rahmen. Sein direktes Elternelement ist `div`; es ist ein Nachfahre, aber kein direktes Kind von `section`.

03 Pseudoklassen und Pseudoelemente nutzen

a) Mauszeiger und Tastaturfokus

```
a { color: #005a7a; text-decoration: underline; }
a:hover { color: #154a6c; background-color: #e6f3fa; }
a:focus { outline: 3px solid #005a7a; outline-offset: 3px; }
```

`:hover` gilt, während der Mauszeiger über dem Link steht. `:focus` gilt, wenn der Link den Eingabefokus besitzt, etwa nach dem Erreichen mit der Tabulatortaste. Beide Zustände können gleichzeitig vorliegen. Ein fokussierter Link lässt sich mit der Eingabetaste öffnen; mit Umschalt und Tab wechseln Sie rückwärts.

Die `outline` macht den Fokus sichtbar, ohne Platz im Layout zu beanspruchen. Der Fokus darf nicht ohne sichtbaren Ersatz entfernt werden.

b) Position unter den Geschwisterelementen

`section > p:first-child` hat **keinen Treffer**: Das erste Kind des Abschnitts ist `h2`, nicht `p`. `section > p:nth-child(2)` wählt dagegen das Element mit der ID `p1` aus.

Gezählt werden alle Geschwisterelemente unabhängig von ihrem Elementtyp: `h2` ist Nummer 1, `p1` Nummer 2, `div` Nummer 3, `p3` Nummer 4 und `a1` Nummer 5. Textknoten und Kommentare zählen nicht mit. `p2` ist kein direktes Kind des Abschnitts.

c) Ein dekoratives Zeichen voranstellen

```
p.info::before {
  content: "• ";
  color: #005a7a;
}
```

`::before` erzeugt ein Pseudoelement vor dem eigentlichen Inhalt des Absatzes. `content` legt seinen erzeugten Inhalt fest. Ohne einen entsprechenden Wert, etwa bei `content: none`, wird dieses Pseudoelement nicht dargestellt.

Eine **Pseudoklasse** beschreibt eine Bedingung für ein vorhandenes Element, etwa Fokus oder Position unter Geschwistern. Ein **Pseudoelement** adressiert einen Teil der Darstellung beziehungsweise erzeugten Inhalt. Der mit `content` erzeugte Punkt wird vor dem Absatztext angezeigt. Dabei wird kein zusätzliches HTML-Element in das DOM eingefügt. Inhaltlich wichtige Informationen gehören ins HTML; hier ist das Zeichen nur Dekoration.

d) Abschnitte mit PDF-Link auswählen

```
section:has(a[href$=".pdf"]) {
  border: 2px solid #005a7a;
}
```

Ausgewählt wird der umschließende Abschnitt, im Beispiel also `section` mit der ID `kurs`. Der Link selbst ist nicht das Ziel der Regel. Das Attributmuster prüft die Zeichenfolge des Linkziels; es erkennt weder den tatsächlichen Dateityp noch beispielsweise `material.pdf?download=1`.

Die Pseudoklasse `:has()` wird in aktuellen Versionen der verbreiteten Browser unterstützt; MDN führt sie seit Dezember 2023 als browserübergreifend verfügbar. In älteren Browsern kann die Unterstützung fehlen. Der Selektor findet den Abschnitt `kurs`; der festgelegte Rahmen wird dargestellt.

Quelle zur Browserunterstützung: MDN zu :has(), abgerufen am 15. Oktober 2026.

04 Kaskade und Spezifität erklären

a) Farbe und Spezifität

Der Absatz ist blau. Unter den direkt auf ihn zutreffenden normalen Regeln hat `#kurs .info` die höchste Spezifität.

Selektor	Spezifität	Wirkung im Beispiel
`main`	`(0, 0, 1)`	Setzt Schwarz am Elternelement; trifft den Absatz nicht direkt.
`p`	`(0, 0, 1)`	Setzt Grau am Absatz.
`.info`	`(0, 1, 0)`	Setzt Grün am Absatz.
`#kurs .info`	`(1, 1, 0)`	Setzt Blau am Absatz und gewinnt.
`p.info`	`(0, 1, 1)`	Setzt Rot, ist aber weniger spezifisch.

Die drei Positionen zählen IDs, Klassen beziehungsweise Attribute beziehungsweise Pseudoklassen sowie Typen beziehungsweise Pseudoelemente. Verglichen wird von links nach rechts. Die zusätzliche ID in `#kurs .info` entscheidet daher vor der Anzahl der Typselektoren.

Kombinatoren wie Leerzeichen, `>`, `+` und `~` tragen nichts zur Spezifität bei. Die Reihenfolge entscheidet erst bei ansonsten gleichem Rang. Eine direkt festgelegte Farbe am Absatz hat Vorrang vor einer vom Elternteil geerbten Farbe.

b) Inline-Deklaration und Wichtigkeit

Mit dem zusätzlichen Attribut wird der Absatz zunächst **violett**:

```
<p class="info" style="color: purple">Nächster Termin</p>
```

Die normale Inline-Deklaration hat hier Vorrang vor den normalen Autorenregeln in den Stylesheets. Ein `style`-Attribut ist kein Selektor und erhält daher kein Tupel aus der Tabelle.

Wird anschließend die Klassenregel so geändert, ist der Absatz **grün**:

```
.info { color: green !important; }
```

Die wichtige Autorendeklaration gewinnt gegen die normalen Autorendeklarationen, einschließlich der normalen Inline-Deklaration. `!important` erhöht nicht die Spezifität; die Wichtigkeit wird in der Kaskade vor der Spezifität berücksichtigt. Die Aussage gilt unter den im Aufgabenblatt genannten Voraussetzungen.

c) Gleiche Spezifität und Vererbung

Die externe Datei `farbe.css` enthält ausschließlich `p { color: blue; }`. Im `head` steht zunächst:

```
<link rel="stylesheet" href="farbe.css">
<style>p { color: red; }</style>
```

Der Absatz ist **rot**, weil die gleich spezifische rote Regel später steht. Werden `style` und `link` vertauscht, ist er **blau**. Maßgeblich ist ihre Reihenfolge im Dokument, nicht die Geschwindigkeit, mit der die CSS-Datei geladen wird.

Bleibt anschließend nur `main { color: black; }` übrig und gibt es keine eigene Farbangabe am Absatz, erbt dieser **Schwarz** von `main`. Die Eigenschaft `color` wird standardmäßig vererbt.

05 Farben und Transparenz untersuchen

a) Gleichwertige Farbangaben und HSL

`#3399cc`, `#39c` und `rgb(51, 153, 204)` beschreiben dieselbe sRGB-Farbe. In der Kurzschreibweise wird jede Hexadezimalziffer verdoppelt: `3` wird zu `33`, `9` zu `99` und `c` zu `cc`. Diese Werte entsprechen dezimal 51, 153 und 204 für Rot, Grün und Blau.

Auch `hsl(200, 60%, 50%)` beschreibt hier dieselbe Farbe: 200 Grad geben den **Farbton** auf dem Farbkreis an, 60 % die **Sättigung** und 50 % die **Helligkeit im HSL-Modell**. Diese Helligkeit ist kein Maß für den wahrgenommenen Kontrast.

b) Alpha und Hintergrund

Der Alpha-Wert `0.5` bedeutet halbtransparent: Bei der üblichen Überlagerung tragen Vordergrund und deckender Hintergrund jeweils zur Hälfte zur sichtbaren Farbe bei. Auf Weiß wirkt die Farbe daher heller als auf Schwarz. `0` ist vollständig transparent, `1` vollständig deckend.

```
.hell { background-color: white; }
.dunkel { background-color: black; }
.probe { width: 10rem; height: 3rem;
         background-color: rgba(51, 153, 204, 0.5); }
```

Je eine Box mit der Klasse `probe` wird in einen hellen und einen dunklen Container gesetzt. `background-color: transparent` lässt den Hintergrund dahinter vollständig durchscheinen. Das Element behält seinen Platz, seine Textfarbe und einen eventuell gesetzten Rahmen; es verschwindet nicht aus dem Layout.

c) Beispiel für gut lesbaren Text

Textfarbe `#154a6c` auf `#ffffff` ergibt ein Kontrastverhältnis von ungefähr **9,42 : 1**. Der Wert ergibt sich aus den relativen Leuchtdichten beider Farben und kann in den Entwicklerwerkzeugen beim Farbfeld des Textelements kontrolliert werden. Er übertrifft die WCAG-Anforderung von 4,5 : 1 für normalen Text auf Stufe AA.

06 Mit relativen Maßeinheiten rechnen

a) Bezugsgrößen und Ergebnisse

Eigenschaft	Rechnung	Ergebnis
`font-size: 1.5em`	1,5 × 20 px vom Elternteil	30 px
`width: 50%`	0,5 × 600 px	300 px Inhalt
`padding: 1em`	1 × 30 px am Element	30 px je Seite
`margin-left: 2rem`	2 × 16 px an der Wurzel	32 px

Bei `font-size` bezieht sich `em` auf die Schriftgröße des Elternteils, damit die eigene Schriftgröße daraus berechnet werden kann. Bei `padding` gilt die bereits berechnete Schriftgröße des Elements. Bei `content-box` kommen die Innenabstände zur Inhaltsbreite hinzu. Die Inhaltsbreite beträgt 300 px. Einschließlich des linken und rechten Innenabstands ist die Box insgesamt 360 px breit.

06 Mit relativen Maßeinheiten rechnen

b) Viewportbezogene Maße

Wert	Viewport 1200 × 800 px	Viewport 800 × 800 px
`10vw`	0,10 × 1200 px = 120 px	0,10 × 800 px = 80 px
`25vh`	0,25 × 800 px = 200 px	0,25 × 800 px = 200 px
`5vmin`	0,05 × 800 px = 40 px	0,05 × 800 px = 40 px

`vw` bezieht sich auf die Viewportbreite, `vh` auf die Höhe und `vmin` auf die kleinere Größe. Hier ändert sich nur `10vw`.

c) Eine andere Wurzelschriftgröße

Nach `html { font-size: 20px; }` bleiben beim Elternteil 20 px Schriftgröße und 600 px Inhaltsbreite unverändert. Das Kind behält deshalb **30 px** Schriftgröße, **300 px** Inhaltsbreite und **30 px** Innenabstand je Seite. Nur `margin-left: 2rem` steigt auf **40 px**.

`rem` ermöglicht eine gemeinsame Größen- und Abstandsskala unabhängig von der Verschachtelung. Ändert sich die Wurzelschriftgröße, passen sich alle davon abhängigen Maße an.

07 Boxmodell und Abstände prüfen

a) Die vier Bereiche

Von innen nach außen folgen **Inhalt**, **Innenabstand** (`padding`), **Rahmen** (`border`) und **Außenabstand** (`margin`). Die folgende Skizze zeigt die horizontalen Bereiche für `content-box`:

```
| margin | border | padding | Inhalt | padding | border | margin |
| 10 px  |  5 px  |  20 px  | 300 px |  20 px  |  5 px  | 10 px  |
```

Die Inhaltsbreite beträgt **300 px**. Mit Rahmen: 300 + 2 × 20 + 2 × 5 = **350 px**. Mit beiden horizontalen Außenabständen: 350 + 2 × 10 = **370 px**.

b) Alternatives Boxmodell und Wirkung der Outline

Bei `box-sizing: border-box` umfasst `width: 300px` Inhalt, Innenabstände und Rahmen. Es bleiben 300 − 2 × 20 − 2 × 5 = **250 px** Inhalt; mit Außenabständen werden **320 px** benötigt.

Größe	`content-box`	`border-box`
Inhaltsbreite	300 px	250 px
Breite einschließlich Rahmen	350 px	300 px
Mit beiden horizontalen Außenabständen	370 px	320 px

`outline: 3px solid red` reserviert keinen Platz und ändert keine dieser Größen. Die Linie kann angrenzende Bereiche überzeichnen. Die Boxmodell-Anzeige bestätigt die Werte.

c) Kollabierende Außenabstände

Der Abstand beträgt **24 px**, nicht 40 px. Bei den vorgegebenen benachbarten Blockabsätzen kollabieren die beiden positiven vertikalen Margins zum größeren Wert: `max(24px, 16px)`. Diese Aussage gilt für die im Aufgabenblatt beschriebene Situation; horizontale Margins kollabieren nicht.

08 Elementfluss und Textumfluss vergleichen

a) Drei Darstellungsarten

```
<span class="box">Eins</span>
<span class="box">Zwei</span>
<span class="box">Drei</span>
```

```
.box {
  width: 120px; height: 60px;
  border: 1px solid #154a6c;
  display: inline;
}
```

Ändern Sie jeweils nur `display`, sodass alle drei Boxen dieselbe Darstellungsart erhalten. Ohne weitere Regel gilt hier `box-sizing: content-box`.

Wert	Wirkung von Breite und Höhe	Anordnung
`inline`	`width` und `height` wirken bei diesen `span`-Elementen nicht.	Im Textfluss; Text kann über mehrere Zeilen umbrechen.
`block`	120 × 60 px Inhalt; mit dem Rahmen 122 × 62 px.	Untereinander; jede Box beginnt eine neue Zeile.
`inline-block`	120 × 60 px Inhalt; mit dem Rahmen 122 × 62 px.	Nebeneinander; bei Platzmangel brechen ganze Boxen um.

Leerraum zwischen den `inline-block`-Elementen erzeugt gewöhnlich einen Abstand. Unter 122 px verfügbarem Platz kann auch eine einzelne Box überlaufen: Die feste Breite schrumpft nicht mit.

b) Das mittlere Element ausblenden

```
.box:nth-child(2) { display: none; }
```

Im gezeigten Test, in dem die drei Boxen direkte Kinder des `body` sind, wird das mittlere Element nicht mehr gerendert. Es nimmt keinen Platz ein; die übrigen Boxen rücken entsprechend zusammen. **Im DOM bleibt das Element vorhanden**, einschließlich seines Textes und seiner Attribute. Wird die Deklaration entfernt, erscheint es wieder.

c) Ein Bild von Text umfließen lassen

```
<img class="foto" src="bilder/lerngruppe.png"
     alt="Drei Studierende lernen gemeinsam">
<p>Hier steht ein längerer Text über die Lerngruppe.</p>
<h2 class="danach">Nächster Abschnitt</h2>
```

```
.foto { width: 120px; height: auto; float: left;
        margin: 0 1rem 1rem 0; }
.danach { clear: both; }
```

Das gefloatete Bild wird aus dem normalen Blockfluss genommen. Die Textzeilen daneben werden verkürzt und fließen um das Bild; unterhalb steht wieder die volle Breite zur Verfügung. `clear: both` verhindert, dass die nächste Überschrift neben einem vorausgehenden Float derselben Formatierungsumgebung steht.

Ein `inline-block` nimmt dagegen als zusammenhängende Box am Inline-Fluss teil. Er ermöglicht keinen über mehrere Textzeilen reichenden Umfluss wie ein Float. Für einen sichtbaren Vergleich muss der Absatz lang genug sein, um mehrere Zeilen zu bilden.

09 Positionen im Browser nachvollziehen

a) Statische und relative Positionierung

Die Grundstruktur kann in ein HTML-Dokument wie in Aufgabe 2 eingesetzt werden:

```
<div class="rahmen">
  <div class="box">Box 1</div>
  <div class="box markiert">Box 2</div>
  <div class="box">Box 3</div>
</div>
```

```
.rahmen { position: relative; border: 2px solid gray; }
.box { min-height: 4rem; padding: 1rem;
       border: 1px solid #154a6c; }
.markiert { position: relative; top: 20px; left: 30px; }
```

Bei `position: static` haben `top` und `left` keine Wirkung. Die mittlere Box steht normal zwischen Box 1 und Box 3. Bei `position: relative` wird ihre Darstellung um **20 px nach unten und 30 px nach rechts** gegenüber dieser Ausgangsposition verschoben.

Der ursprüngliche Platz bleibt dabei reserviert. Box 3 wird so angeordnet, als wäre Box 2 nicht verschoben. Dadurch können sich die sichtbaren Boxen überlappen; die Verschiebung erzeugt keinen zusätzlichen Platz.

b) Absolute Positionierung im Container

Die Regel für die mittlere Box wird ersetzt durch:

```
.markiert {
  position: absolute;
  top: 0; right: 0; left: auto;
  width: 8rem;
  box-sizing: border-box;
}
```

Die **Paddingbox** von `.rahmen` ist der Bezug, weil dieser Container der nächste positionierte Vorfahr ist. Die Box liegt oben rechts in diesem Bereich. `left: auto` setzt den bisherigen linken Versatz zurück. Durch `border-box` ist die gesamte Rahmenbox 8 rem breit.

Die absolut positionierte Box wird aus dem normalen Fluss genommen. Ihr bisheriger Platz entfällt; Box 3 folgt direkt auf Box 1. Sie vergrößert die automatisch ermittelte Höhe des Containers nicht. Überlappungen sind möglich.

c) Ein Link an einer festen Fensterposition

```
<!-- Am Anfang des body: -->
<h1 id="oben">Positionierung testen</h1>
<!-- Nach den übrigen Inhalten: -->
<a class="nach-oben" href="#oben">Nach oben</a>
```

```
body { min-height: 200vh; }
.nach-oben { position: fixed; bottom: 1rem; right: 1rem;
             padding: 0.5rem; background-color: white; }
```

In dieser einfachen Testseite bleibt der Link beim Scrollen am unteren rechten Rand des Viewports. Er reserviert keinen Platz im Fluss. Die absolute Box aus b) bleibt dagegen am Container ausgerichtet und scrollt mit ihm. Bestimmte Eigenschaften von Vorfahren, etwa `transform`, können den Bezug einer festen Position ändern; sie kommen hier nicht vor.

09 Positionen im Browser nachvollziehen

d) Eine haftende Navigation

Für einen getrennten Test wird der folgende Inhalt verwendet. Die Höhe nach dem Container erlaubt es, über dessen Ende hinauszuscrollen:

```
<h1>Haftende Navigation</h1>
<div class="lang">
  <nav class="haftend"><a href="#ziel">Zum Ziel</a></nav>
  <p>Dieser Bereich steht am Anfang des Containers.</p>
  <div class="abstand"></div>
  <h2 id="ziel">Ende des Containers</h2>
</div>
<div class="nachlauf">Inhalt nach dem Container</div>
```

```
body { margin: 0; }
.lang { border: 2px solid gray; }
.haftend { position: sticky; top: 0;
           padding: 1rem; background-color: #e6f3fa; }
.abstand { height: 150vh; }
.nachlauf { height: 100vh; }
```

Die Navigation nimmt zunächst ihren normalen Platz am Anfang des Containers ein. Sobald sie beim Scrollen den oberen Rand des Scrollbereichs erreichen würde, hält `top: 0` sie dort fest. In diesem Beispiel ist der Scrollbereich der Viewport.

Sie bleibt dabei Teil des normalen Flusses. Ihr Bewegungsbereich ist durch ihren umschließenden Block begrenzt: Nähert sich dessen unteres Ende, wird die Navigation mit ihm nach oben geschoben. Sie kann daher nicht dauerhaft außerhalb von `.lang` am Fensterrand stehen bleiben.

Für einen aussagekräftigen Test muss der Container höher als die Navigation sein. Ein Vorfahr mit eigenem Scrollbereich, etwa durch `overflow: auto`, kann den maßgeblichen Scrollbereich ändern. Die einfachen Testregeln enthalten deshalb keine solche Einstellung.

Positionierung	Platz im normalen Fluss	Bezug im gezeigten Beispiel
`static`	Bleibt erhalten.	Normale Anordnung; Versatzangaben wirken nicht.
`relative`	Bleibt an der Ausgangsposition erhalten.	Eigene normale Position.
`absolute`	Entfällt.	Paddingbox des positionierten Containers.
`fixed`	Entfällt.	Viewport.
`sticky`	Bleibt erhalten.	Scrollbereich, begrenzt durch den umschließenden Block.

Erwartete Beobachtung: Beim Scrollen bleibt der feste Link unabhängig vom Ende des Containers im Viewport. Die haftende Navigation wird dagegen spätestens mit dem Ende ihres Containers weitergeschoben. Damit unterscheiden sich `fixed` und `sticky` auch dann, wenn beide zeitweise am Fensterrand sichtbar sind.

10 Die Webseite der Lerngruppe gestalten

a) Gemeinsame Grundlage für beide Seiten

Beide Dateien binden `css/styles.css` wie in Aufgabe 1 ein. Im `head` steht zusätzlich die folgende Angabe, damit mobile Browser die tatsächliche Gerätebreite als Ausgangspunkt verwenden:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Der folgende erste Teil von `css/styles.css` legt die gemeinsame Gestaltung fest:

```
* { box-sizing: border-box; }
html { font-size: 100%; }
body {
  width: calc(100% - 2rem);
  max-width: 60rem;
  margin: 2rem auto;
  font-family: Arial, Helvetica, sans-serif;
  line-height: 1.5;
  color: #1b2e44;
  background-color: #f4f7fb;
  overflow-wrap: anywhere;
}
header, main, footer {
  padding: 1rem;
  border: 1px solid #b8c9d6;
  background-color: white;
}
h1, h2 { color: #154a6c; }
h1 { font-size: 2rem; }
h2 { font-size: 1.5rem; }
```

`max-width` begrenzt die Breite. Die automatischen horizontalen Außenabstände zentrieren den Inhalt. `calc(100% - 2rem)` lässt bei schmalem Fenster je 1 rem am linken und rechten Rand. Die Schriftliste enthält Ersatzschriften; `sans-serif` ist die allgemeine Rückfallebene.

b) Navigation und Bild

Die gemeinsame CSS-Datei wird um diese Regeln erweitert:

```
nav { margin: 1rem 0; }
nav a {
  display: inline-block;
  padding: 0.5rem 0.75rem;
  margin: 0.25rem 0.25rem 0.25rem 0;
  max-width: 100%;
  background-color: #e6f3fa;
}
a { color: #005a7a; text-decoration: underline; }
a:hover { color: #154a6c; background-color: #e6f3fa; }
a:focus { outline: 3px solid #005a7a; outline-offset: 3px; }
img { display: block; max-width: 100%; height: auto; }
```

Die Navigationslinks stehen im Inline-Fluss und dürfen als ganze Boxen umbrechen. `max-width` und der geerbte Textumbruch verhindern überbreite Links. Das Bild wird höchstens so breit wie sein Container; `height: auto` erhält das Seitenverhältnis.

10 Die Webseite der Lerngruppe gestalten

c) Kontaktabschnitt und sichtbarer Hinweis

In `index.html` erhält der vorhandene Kontaktabschnitt zusätzlich die Klasse `kontakt`. Seine ID bleibt für Links auf `#kontakt` erhalten:

```
<section id="kontakt" class="kontakt">
  <p class="termin">Nächstes Treffen</p>
  <h2>Kontakt</h2>
  <p>Die Lerngruppe ist über das Forum des Kurses erreichbar.</p>
</section>
```

Den letzten Teil der gemeinsamen CSS-Datei bilden diese Regeln:

```
.kontakt {
  position: relative;
  margin-top: 1.5rem;
  padding: 4.5rem 1rem 1rem;
  border: 2px solid #154a6c;
  background-color: #edf6fa;
}
.termin {
  position: absolute;
  top: 1rem; right: 1rem;
  margin: 0;
  padding: 0.25rem 0.5rem;
  max-width: calc(100% - 2rem);
  font-size: 0.875rem;
  line-height: 1.4;
  color: white;
  background-color: #154a6c;
}
```

Der Kontaktabschnitt liefert den Positionierungsbezug. Der obere Innenabstand von 4,5 rem reserviert Raum für den kurzen Hinweis. Dieser steht als echter Text im HTML und wird nicht mit CSS erzeugt. Bei einem längeren oder mehrzeiligen Hinweis muss der reservierte Platz erneut geprüft werden.

d) Prüfung und drei Gestaltungsentscheidungen

Bei ungefähr **375 px** Breite sollen die Navigationslinks umbrechen, das Bild im Container bleiben und die Inhalte ohne horizontales Scrollen lesbar sein. Bei **1200 px** soll der Inhalt zentriert bleiben und höchstens 60 rem breit werden. Bei einer Wurzelschriftgröße von 16 px sind dies 960 px.

Bei **200 % Browserzoom** steht für dieselbe Fenstergröße weniger Platz in CSS-Pixeln zur Verfügung. Prüfen Sie beide Seiten erneut auf Umbrüche, horizontales Scrollen und Überlappungen, besonders beim Hinweis. Alle Links müssen mit Tab erreichbar, sichtbar fokussiert und mit der Eingabetaste aktivierbar sein. Das sind die Prüfkriterien; die konkreten Beobachtungen hängen von Browser, Fenster und Inhalt ab.

Entscheidung 1: `max-width: 60rem` hält Textzeilen überschaubar. `width: calc(100% - 2rem)` bewahrt Randabstände auch auf schmalen Ansichten.

Entscheidung 2: `nav a { display: inline-block; }` verbindet gut nutzbare Klickflächen mit natürlichem Umbruch. Die separate Regel `a:focus` macht die Tastaturposition erkennbar.

Entscheidung 3: `.kontakt { position: relative; }` bindet den absolut positionierten Hinweis an den Abschnitt. Der obere Wert in `padding: 4.5rem 1rem 1rem` verhindert beim vorgesehenen kurzen Text, dass der Hinweis die Überschrift verdeckt.